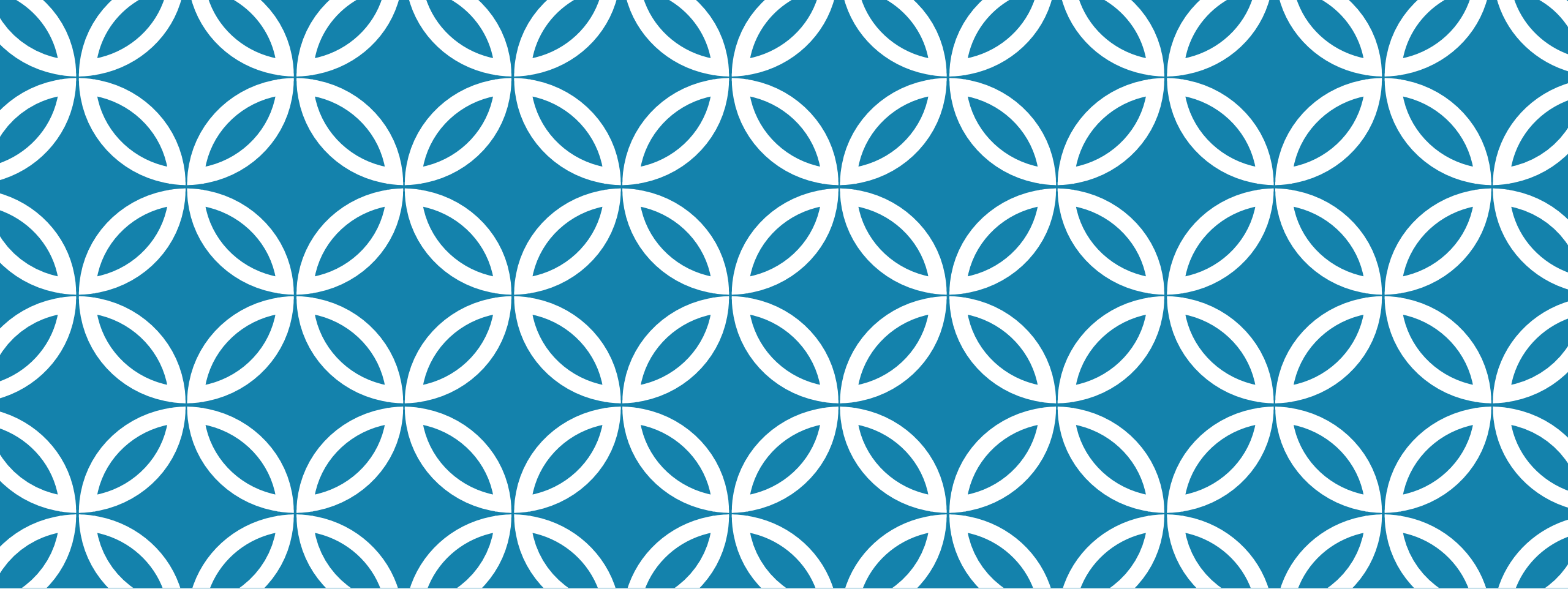




STATIC PROGRAM ANALYSIS

Lecture 2 – Structural Problems

4-STEP ANALYSIS PROCESS

1. Define a problem
2. State as a decision problem $\Rightarrow$ what information to compute to answer it
3. Instantiate an analysis to compute information
4. Examine results

PROBLEMS OVER PROGRAM STRUCTURE

- Examining a program's text – the highest level of abstraction
- Mainly focuses on ``pattern`` matching of the text
- Very quick and efficient
- Answers limited number of questions, and more not that well

1. Define a problem:

- A. How large is a program?
- B. Is a program branch heavy?
- C. Does it have complex branches?

STEP2: STATE AS A DECISION PROBLEM: WHAT INFORMATION TO COMPUTE

- A. Define N threshold to quantify “large”; compute the number of lines M in a code; if $M > N$ then the answer is yes.
- B. Define B threshold to quantify “too many”; compute the number C of if statements; if $C > B$ then the answer is yes.
- C. Does a program have if/else/then branch, or a nested conditional statement?

STEP3: INSTANTIATE ANALYSIS TO COMPUTE INFORMATION

- Pattern matching with grep and or your own tool
- What are those patterns
 1. New lines
 2. `.*if.*`
 3. `.*if.*else.*`
- Use pattern finding program, or write you own text reader (?)

Let's try it in-class

STEP4: EXAMINE RESULTS

1. Check for grep/matching would give you.
2. Check what code processing would give us.
3. Check results with hand analyzed programs.

Any problems with the current abstraction?

OTHER PROBLEMS

1. Is a program well-documented?
2. Other text-related properties?

PRECISION OF ANALYSIS

- How to differentiate between comment and non-comment lines?
- ``if`` keyword in a comment vs text
- Processing as a text is not precise enough – can produce false positive results
- Use more precise abstraction: Abstract Syntax Tree (AST)
- FindBugs uses pattern matching but on AST level
 - <http://findbugs.sourceforge.net/bugDescriptions.html>

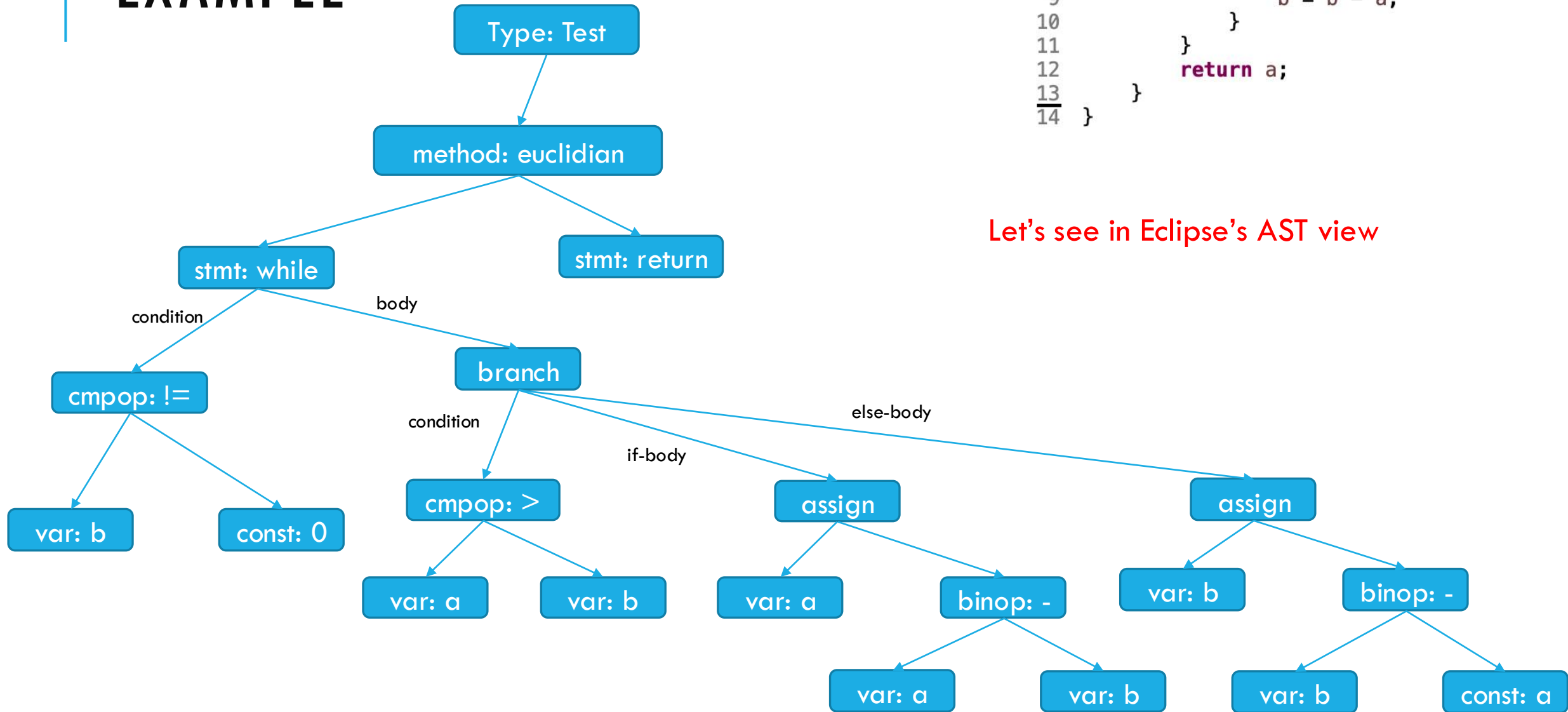
ABSTRACT SYNTAX TREE

- Using recursive decent parser construct parse tree from the language's grammar
- Apply contextual analysis to assign additional meaning to nodes and remove inessential syntax elements such as delimiters, braces
- We assume program correctness (compliant), but compiler use AST to perform semantic analysis on the code to ensure program correctness, e.g., types matching, operator overloading, something that cannot be done by the grammar
- AST data-structure is further analyzed
 - Translate to a different intermediate representation
 - Modified to change the original code
 - Structured merging based on AST differences – avoids spurious conflicts
 - Close detection
 - IDEs use AST to perform refactoring, some

EXAMPLE

```
1 public class Test {  
2  
3  
4 public int ecludian(int a, int b) {  
5     while (b != 0) {  
6         if (a > b) {  
7             a = a - b;  
8         } else {  
9             b = b - a;  
10        }  
11    }  
12    return a;  
13 }  
14 }
```

Let's see in Eclipse's AST view





NEXT TIME

- Traverse AST using visitors
- Wala static analyzer
 - Look at its front-end AST, which is Eclipse's plug-in
 - Solving problems